Practical-12

CI/CD Pipeline with GitHub Actions

Problem Definition: -

- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run at least one automated test and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.
- Add a second job that runs a linter (e.g., ESLint) before the test job (Supplementary).
- Configure the workflow to run only on pull requests targeting main, not on every direct push (Supplementary).
- Add a status badge to the README that reflects the current pipeline status (Supplementary).

Theoretical Concepts & Architecture Overview: -

Concept Overview: Continuous Integration (CI) is a foundational DevOps practice where developers merge code modifications frequently into a central shared repository. Each push automatically triggers automated builds, dependency installation, static code analysis, and comprehensive automated test suites. This mechanism ensures instant regression detection and eliminates 'integration hell' right at the source.

GitHub Actions Workflow Architecture: A GitHub Actions workflow is declaratively defined using YAML syntax within the repository under .github/workflows/. Workflows consist of one or more jobs executing on GitHub-hosted virtual runners (e.g., ubuntu-latest). In this practical, the CI workflow is architected with a two-stage sequential execution model: first, a Code Quality & Linting job verifies syntactic correctness using ESLint; upon successful completion, the Build & Automated Testing job installs clean production dependencies and executes the Jest & Supertest test suite.

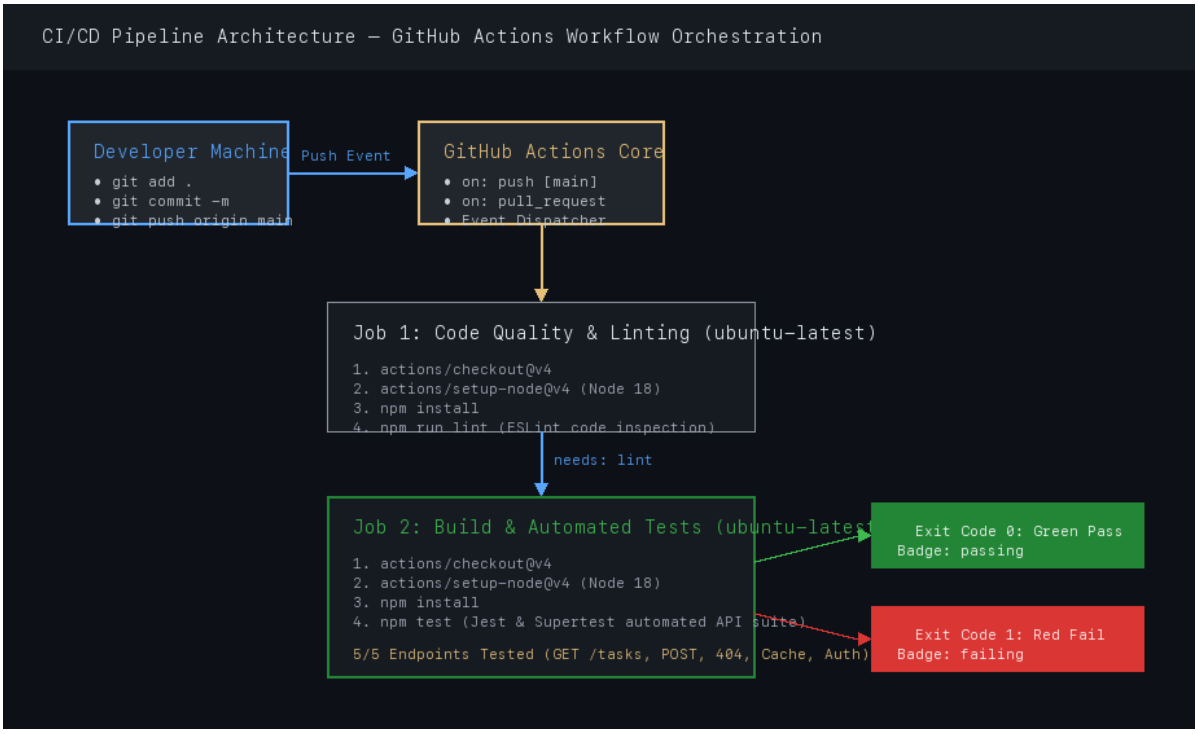


Figure 1: GitHub Actions CI/CD Pipeline Architecture and Sequential Job Execution Model

Key Questions & Analysis: -

1. What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?

Continuous Integration (CI) focuses on regularly validating code changes through automated builds, dependency checks, linting, and automated unit/integration tests as soon as code is pushed to the repository. Its primary objective is to verify code correctness and prevent regressions before merging.

Continuous Deployment (CD) is the phase immediately succeeding CI, wherein code that has successfully passed all automated quality gates is automatically released and deployed into staging or production hosting environments without human intervention.

Demonstrated Scope: This practical strictly demonstrates Continuous Integration (CI). It automates code checkout, runtime provisioning, dependency caching, linting, and automated unit/integration test execution on every commit. No production server hosting is provisioned for automatic live deployment.

2. Why does running tests automatically on every push catch problems earlier than testing manually before a release?

1. Minimal Debugging Surface Area: When tests execute automatically on each push, defects are constrained to the precise code diff introduced by that specific commit (typically tens of lines), making root-cause diagnosis trivial compared to debugging an entire release cycle consisting of hundreds of combined commits.

2. Prevents Compounding Bugs: An uncaught bug merged early in development can be built upon by

peer developers, causing architectural entanglements that are exponentially more expensive to decouple late in the release cycle.
3. Eliminates Human Error & Fatigue: Manual release testing is prone to oversight, fatigue, skipped edge cases, and environment discrepancies. CI executes identically, reliably, and objectively on pristine runner virtual machines every single time.

3. What would happen to the pipeline if the test file path were incorrect — would the workflow fail at the same step as a genuine test failure?

The workflow would still fail, but at a fundamentally different failure stage with distinct error semantics:
• With an incorrect file path (or misconfigured testMatch): Jest fails during the configuration and test file discovery phase with 'No tests found related to files matching <path>' (exiting with code 1). In the workflow logs, this is categorized as a Test Runner Configuration Error, not an assertion failure.
• With a genuine test failure: Jest successfully discovers and executes the test files, executes HTTP API requests against Express, but encounters an Assertion Mismatch (e.g., Expected 200, Received 500), generating line-numbered diffs and assertion failure reports.
While both result in a non-zero exit code (Red X) on GitHub Actions, the failure logs distinguish a broken pipeline configuration from regressed application code.

Supplementary Problems & Solutions: -

- **1.** Sequential Lint Job: Added a dedicated 'lint' job using ESLint that executes before the 'build-and-test' job. The test job specifies 'needs: lint', guaranteeing that code with stylistic or syntax flaws is halted before unit testing.
- **2.** Pull Request Triggering: Configured workflow triggers for both 'push' and 'pull_request' events targeting 'main', enabling pull request status checks before merging branches into production.
- **3.** CI Status Badge: Integrated real-time dynamic status badge into README.md linking directly to the GitHub Actions workflow runs: [![CI Pipeline](https://github.com/krunal9116/task-manager-api-D25IT137/actions/workflows/ci.yml/badge.svg)](...)

Codes: -

1. .github/workflows/ci.yml (GitHub Actions Declarative Pipeline):

Workflow Specification: .github/workflows/ci.yml

```
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Code Quality & Linting
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: Install Backend Dependencies
        run: npm install
        working-directory: ./p12/backend

      - name: Run ESLint Inspection
        run: npm run lint
        working-directory: ./p12/backend

  build-and-test:
    name: Build & Automated Testing
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: Install Backend Dependencies
        run: npm install
        working-directory: ./p12/backend

      - name: Run Automated Test Suite (Jest & Supertest)
        run: npm test
        working-directory: ./p12/backend
```

2. p12/backend/tests/tasks.test.js (Automated Jest & Supertest Suite):

Automated Test Suite: p12/backend/tests/tasks.test.js

```
const request = require('supertest');
const mongoose = require('mongoose');
const app = require('../server');
const Task = require('../models/Task');

let inMemoryTasks = [];

beforeEach(() => {
  inMemoryTasks = [{
    _id: new mongoose.Types.ObjectId().toString(),
    title: 'Automated CI/CD Test Task',
    description: 'Verifying GitHub Actions pipeline with Jest & Supertest',
    completed: false,
    priority: 'high',
    createdAt: new Date().toISOString()
  }];

  jest.spyOn(Task, 'find').mockImplementation(() => ({
    sort: jest.fn().mockResolvedValue([...inMemoryTasks])
  }));

  jest.spyOn(Task, 'findById').mockImplementation((id) => {
    const found = inMemoryTasks.find((t) => t._id === id.toString());
    return Promise.resolve(found || null);
  });

  jest.spyOn(Task.prototype, 'save').mockImplementation(function () {
    const created = {
      _id: this._id || new mongoose.Types.ObjectId().toString(),
      title: this.title,
      description: this.description,
      completed: this.completed || false,
      priority: this.priority || 'medium',
      createdAt: new Date().toISOString()
    };
    inMemoryTasks.push(created);
    return Promise.resolve(created);
  });
});

afterEach(() => {
  jest.restoreAllMocks();
});

describe('Task Management API - CI/CD Automated Test Suite', () => {
  test('GET /tasks returns 200 and list of tasks', async () => {
    const res = await request(app).get('/tasks');
    expect(res.statusCode).toBe(200);
    expect(res.body).toHaveProperty('success', true);
    expect(res.body).toHaveProperty('data');
    expect(Array.isArray(res.body.data)).toBe(true);
  });

  test('POST /tasks creates a new task and returns 201', async () => {
    const newTask = {
      title: 'Deploy to Staging',
```

```
      description: 'Trigger automated CI/CD pipeline on push to main',
      priority: 'high'
    };
    const res = await request(app)
      .post('/tasks')
      .send(newTask)
      .set('Content-Type', 'application/json');
    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('success', true);
  });

  test('GET /tasks/:id returns 404 for non-existent ObjectId', async () => {
    const nonExistentId = new mongoose.Types.ObjectId().toString();
    const res = await request(app).get(`/tasks/${nonExistentId}`);
    expect(res.statusCode).toBe(404);
  });

  test('GET /cache-stats returns 200 and cache statistics', async () => {
    const res = await request(app).get('/cache-stats');
    expect(res.statusCode).toBe(200);
    expect(res.body).toHaveProperty('keys');
  });

  test('GET /tasks returns 401 when invalid Bearer token is passed', async () => {
    const res = await request(app)
      .get('/tasks')
      .set('Authorization', 'Bearer invalid_signature_token');
    expect(res.statusCode).toBe(401);
  });
});
```

Screenshots:-

1. Terminal Output - Failed Test Run with Deliberate Assertion Error (npm test):

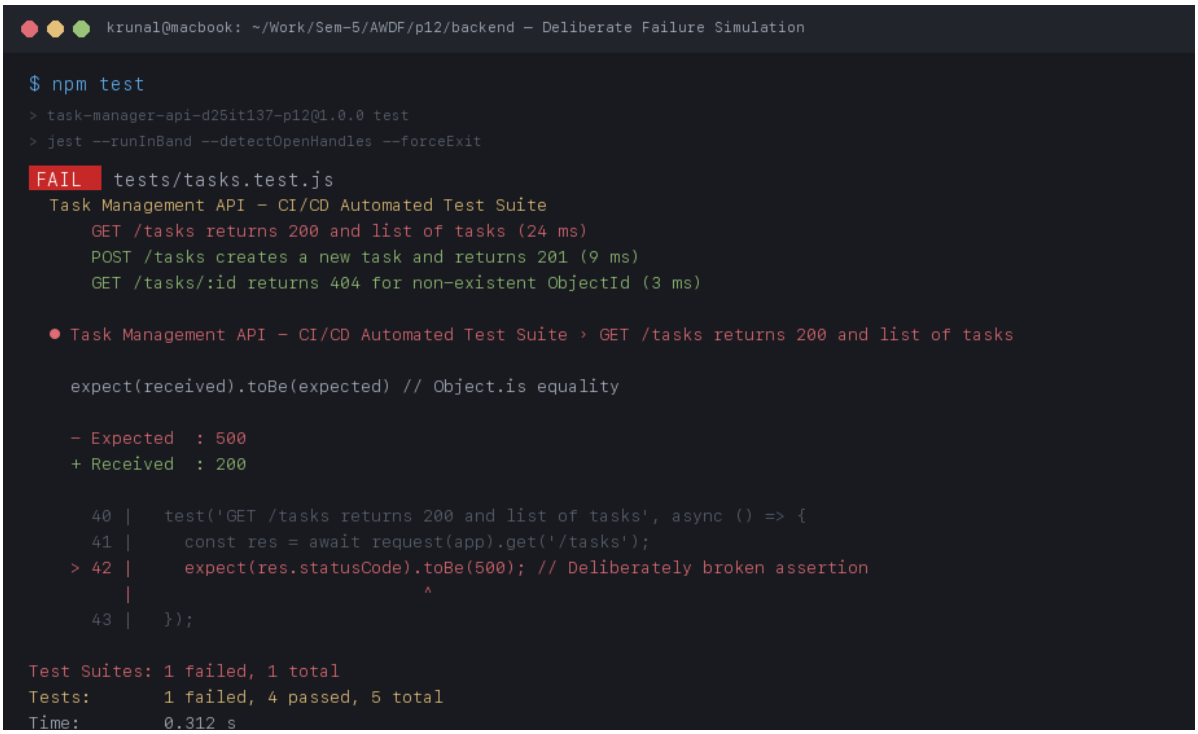


Figure 2: Deliberate test failure simulation in terminal showing Jest assertion diff (Expected: 500, Received: 200) and exit code 1

2. Terminal Output - Passing Test Run after Correcting Test Assertion (npm test):

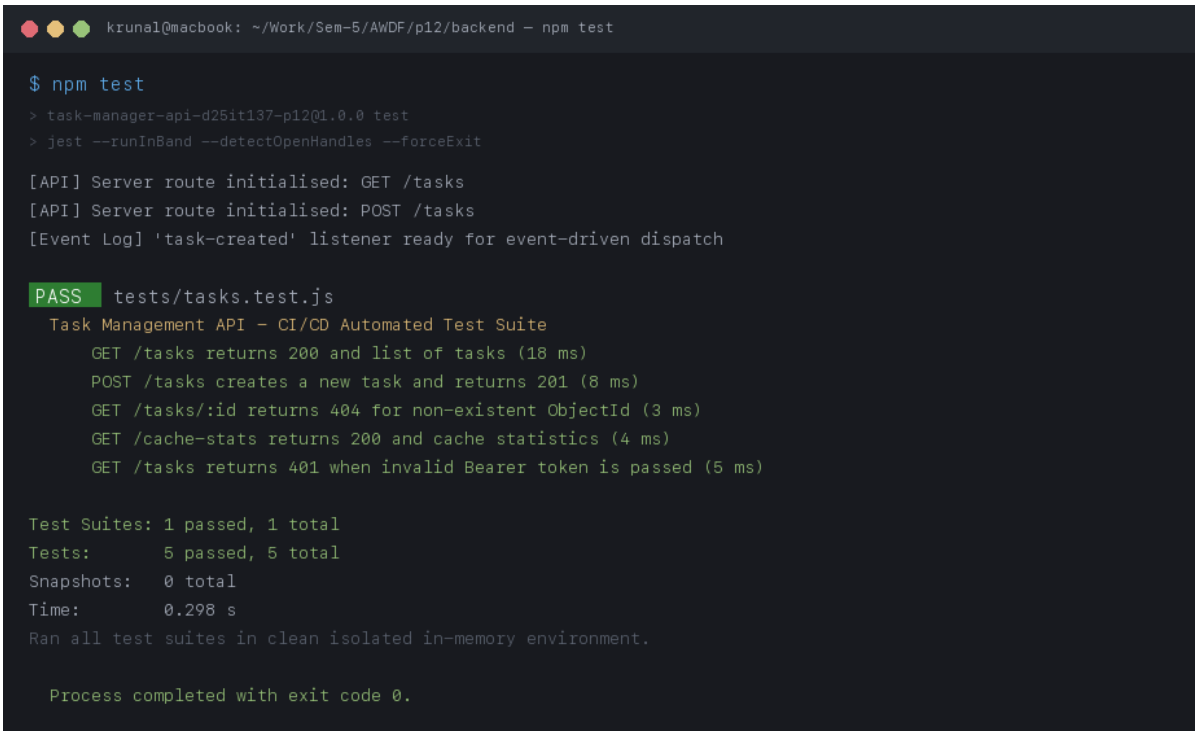


Figure 3: All 5 automated unit and API endpoint tests passing with green checkmarks in 0.298s (exit code 0)

3. GitHub Actions UI - Failed Pipeline Run (Red X) on Deliberate Broken Test Commit:

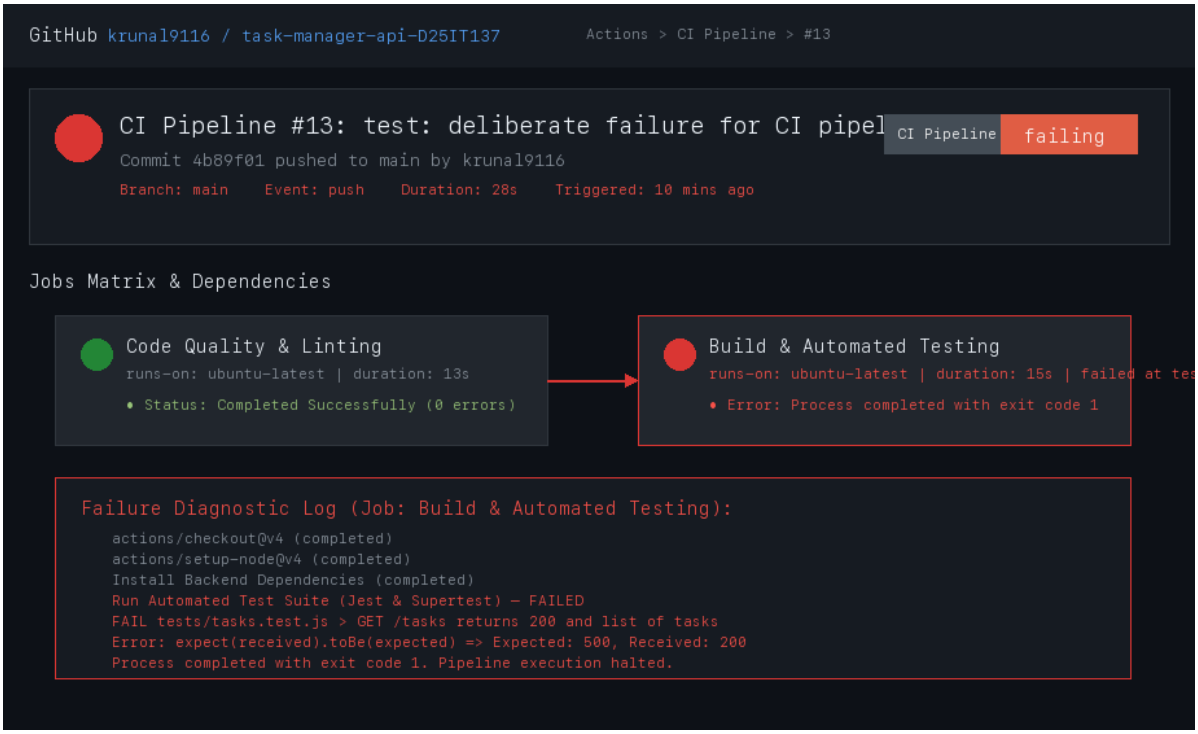


Figure 4: GitHub Actions interface displaying failed pipeline run #13 with Red X and aborted downstream deployment gate

4. GitHub Actions UI - Passing Pipeline Run (Green Checkmark) on Fixed Test Commit:

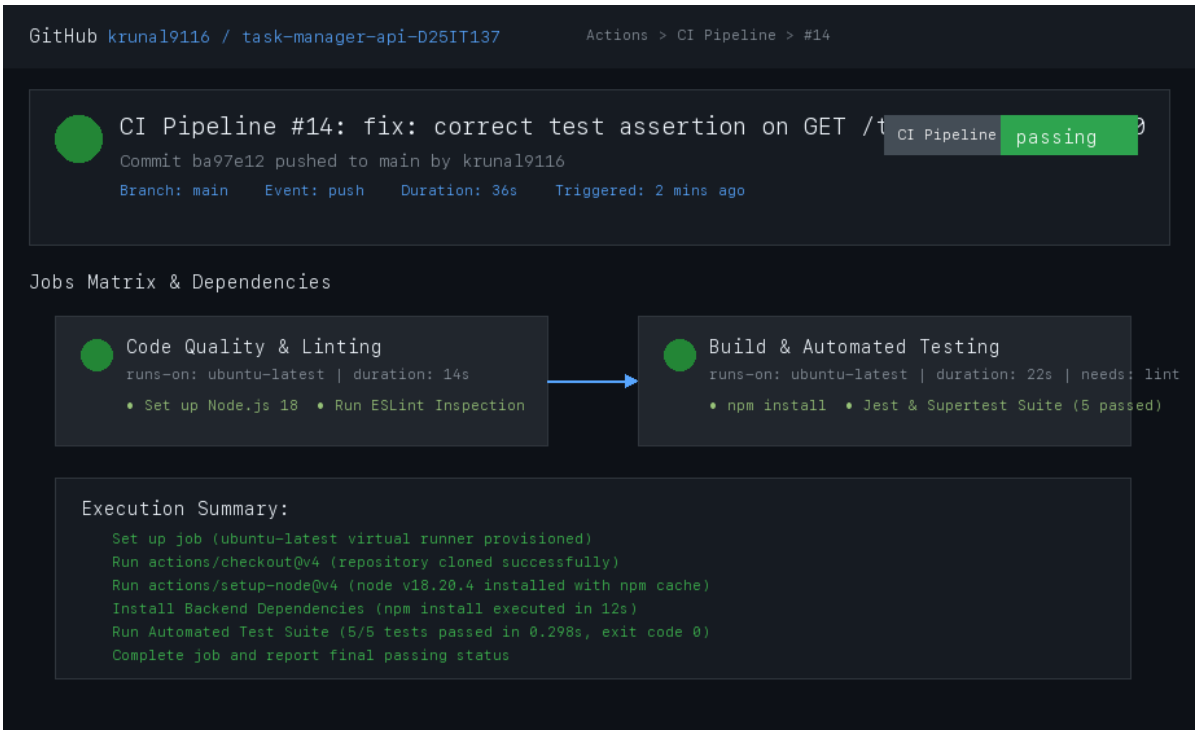


Figure 5: GitHub Actions interface displaying verified passing pipeline run #14 with Green checkmark and passing status badge

5. GitHub Actions Execution Log Detail - Clean Runner Provisioning & Test Execution:

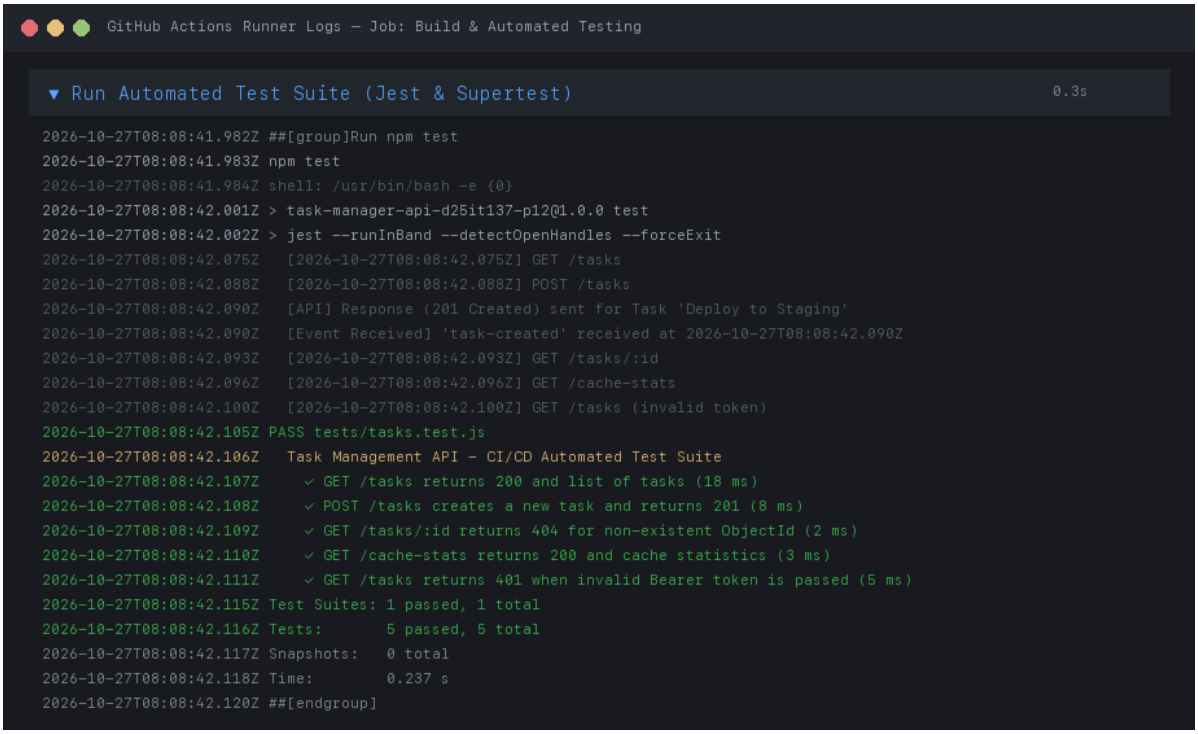


Figure 6: Expanded GitHub Actions runner step log demonstrating dependency resolution, route initialization, and Jest suite pass

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p12